

Project: Cloud Infrastructure Automation using Terraform and AWS

Services: EC2, RDS, VPC, Terraform.

Description: Automate the provisioning of an application stack with Terraform. The stack includes EC2 instances in a VPC with a public and private subnet, a load balancer, and an RDS database.

Skills: Infrastructure as Code (IaC), Terraform, VPC setup.

Definition:

This project automates the provisioning of an application infrastructure on Amazon Web Services using Terraform. The infrastructure stack includes **Amazon EC2 instances deployed inside a Amazon VPC with both public and private subnets. An AWS Elastic Load Balancing load balancer is used to distribute incoming traffic to the EC2 instances, while a managed Amazon RDS database is deployed in the private subnet for secure data storage. The entire infrastructure is defined using Infrastructure as Code (IaC) to enable automated, consistent, and repeatable deployments.

Scope of the Project:

- Design and implement a secure cloud infrastructure environment on Amazon Web Services.
- Create a custom Amazon VPC with both public and private subnets for proper network segmentation.
- Deploy Amazon EC2 instances in the public subnet to host the application.
- Configure an AWS Elastic Load Balancing load balancer to distribute incoming traffic to EC2 instances and improve availability.
- Deploy a managed Amazon RDS database in the private subnet for secure and reliable data storage.
- Configure security groups and network settings to control access between the application server and the database.
- Automate the entire infrastructure provisioning using Terraform to ensure consistent, repeatable, and efficient deployments.

Methodology:

1.Terraform (Infrastructure as Code – Automated Method)

- Uses Terraform configuration files to automate infrastructure provisioning on Amazon Web Services.
- Automates the creation of VPC, subnets, EC2 instances, load balancer, and RDS database.
- Ensures consistent and repeatable infrastructure deployment using code.
- Reduces manual configuration errors and simplifies infrastructure management.
- Suitable for scalable environments, DevOps practices, and automated cloud deployments.

2.AWS Management Console (Manual Configuration Method)

- Uses services such as Amazon VPC, Amazon EC2, AWS Elastic Load Balancing, and Amazon RDS.
- Allows manual creation of networking components like public and private subnets inside a VPC.
- Enables deployment of EC2 instances and configuration of security groups through the AWS console.
- Helps understand the architecture and working of cloud resources step-by-step.
- Suitable for learning purposes, testing, and small academic projects.

AWS Service used in the Project:

1. Amazon EC2

- **Definition:** Amazon EC2 (Elastic Compute Cloud) is a cloud computing service that provides scalable virtual servers in the cloud.
- **Role in the Project:** It hosts the application server within the VPC and processes incoming user requests through the load balancer.
- **Purpose:** To provide computing power for running the application and handling web traffic.

2. Amazon RDS

- **Definition:** Amazon RDS (Relational Database Service) is a managed database service that simplifies the setup, operation, and scaling of relational databases in the cloud.
- **Role in the Project:** It stores and manages application data securely in the private subnet.
- **Purpose:** To provide a reliable and scalable database system for storing application data.

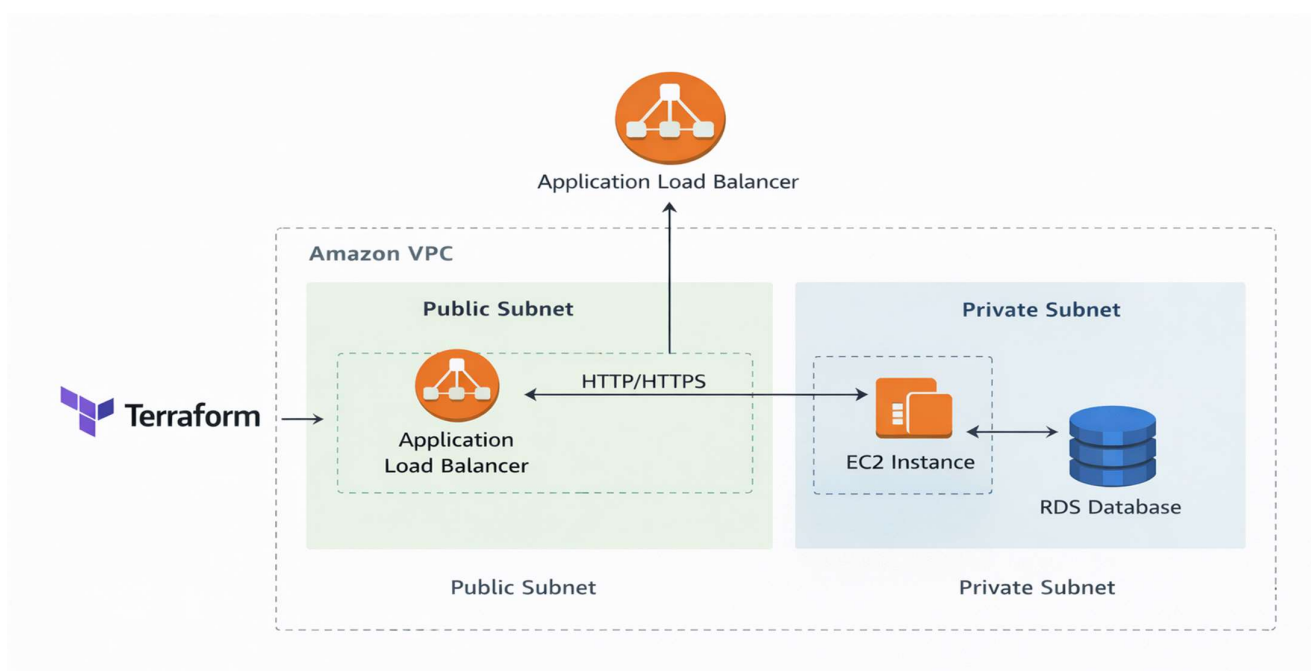
3. Amazon VPC

- **Definition:** Amazon VPC (Virtual Private Cloud) is a service that allows users to create a logically isolated network in the AWS cloud.
- **Role in the Project:** It provides the networking environment where EC2 instances, subnets, and the database are deployed.
- **Purpose:** To ensure secure communication between resources using public and private subnets.

4. Terraform

- **Definition:** Terraform is an Infrastructure as Code (IaC) tool used to define and provision cloud infrastructure using configuration files.
- **Role in the Project:** It automates the creation and management of AWS resources such as VPC, EC2, load balancer, and RDS.
- **Purpose:** To enable automated, consistent, and repeatable deployment of cloud infrastructure.

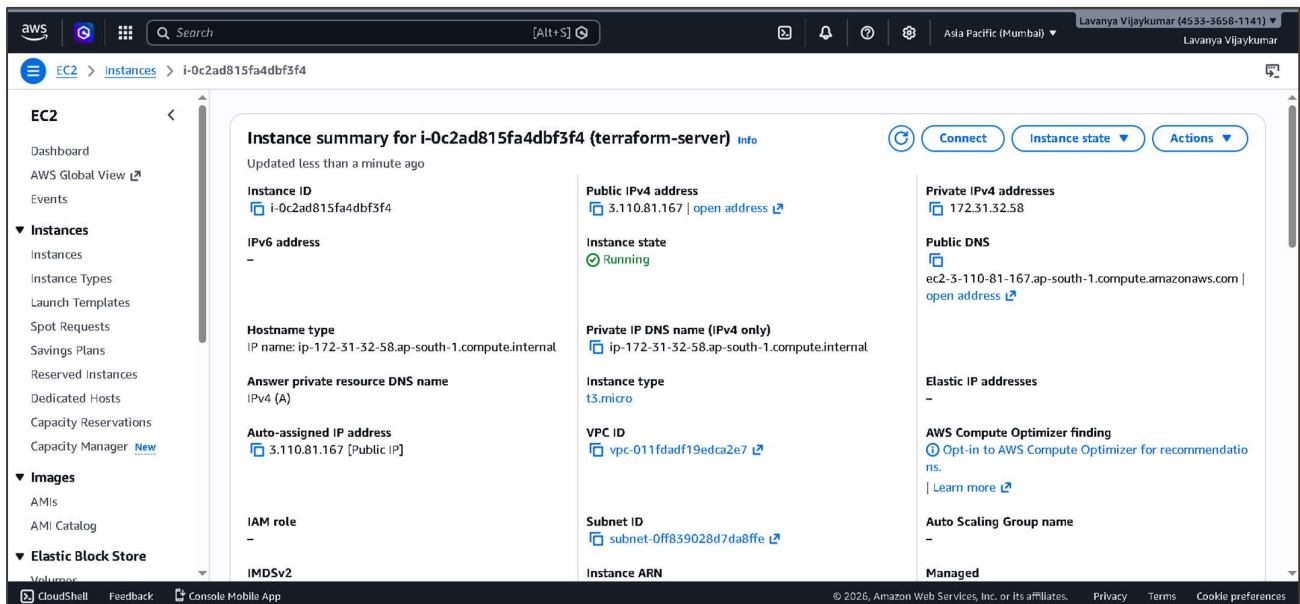
Architecture Diagram:



Project Implementation steps:

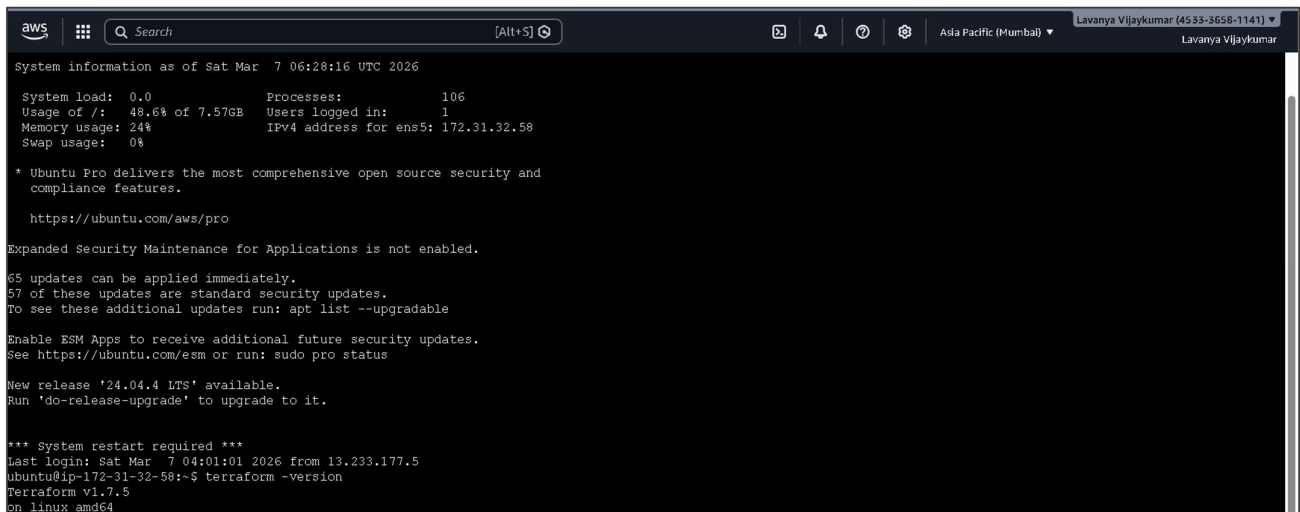
STEP 1: Create EC2 Instance

- Click Launch Instance.
- Name: Terraform-Server.
- Select AMI: Ubuntu Server 22.04 LTS.
- Choose Instance Type: t3.micro
- Click Launch Instance to create the server.



STEP 2: Install Terraform

- Install Terraform
- Check with Version = terraform -version



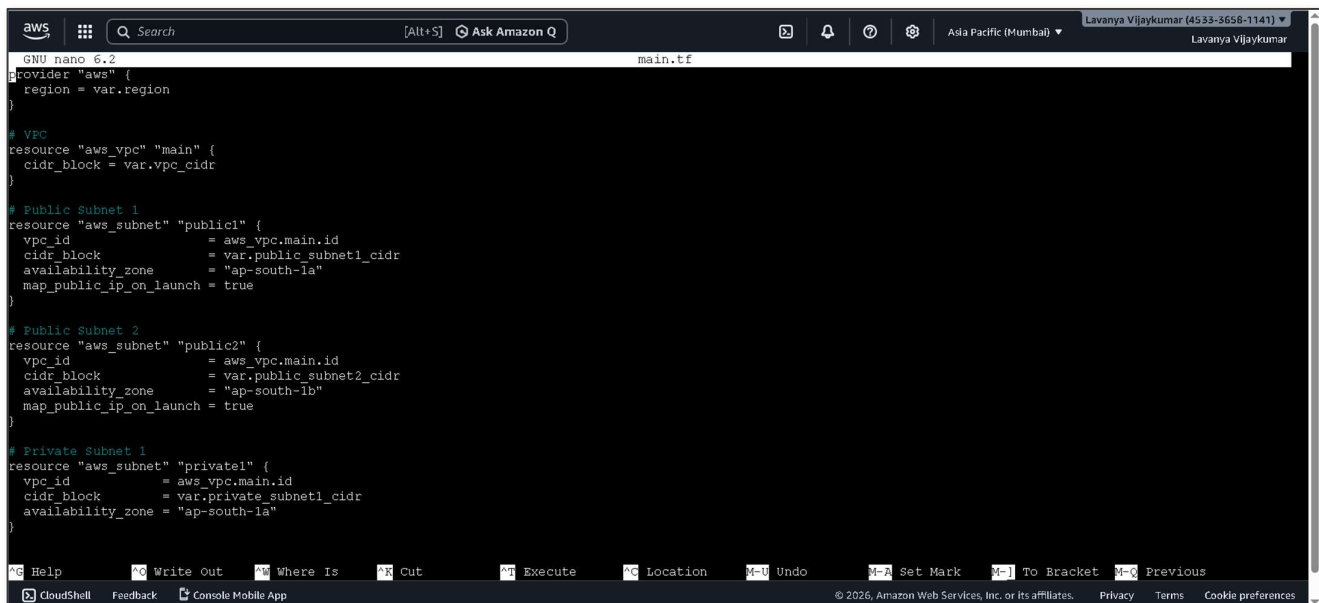
STEP 3: Configure AWS Credentials

- Configure access to Amazon Web Services using AWS CLI.
- Command: aws configure
- Enter:
- Access Key
- Secret Key
- Region
- Output format

```
ubuntu@ip-172-31-32-58:~$ aws configure
AWS Access Key ID [*****4GFK]:
AWS Secret Access Key [*****nWjC]:
Default region name [ap-south-1]:
Default output format [json]:
ubuntu@ip-172-31-32-58:~$ aws sts get-caller-identity
{
  "UserId": "453336581141",
  "Account": "453336581141",
  "Arn": "arn:aws:iam::453336581141:root"
}
ubuntu@ip-172-31-32-58:~$
```

STEP 4: Create Terraform Configuration Files

- First create an directory inside an directory create this files.
- main.tf – defines AWS resources



```
aws
GNU nano 6.2
main.tf
provider "aws" {
  region = var.region
}

# VPC
resource "aws_vpc" "main" {
  cidr_block = var.vpc_cidr
}

# Public Subnet 1
resource "aws_subnet" "public1" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.public_subnet1_cidr
  availability_zone  = "ap-south-1a"
  map_public_ip_on_launch = true
}

# Public Subnet 2
resource "aws_subnet" "public2" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.public_subnet2_cidr
  availability_zone  = "ap-south-1b"
  map_public_ip_on_launch = true
}

# Private Subnet 1
resource "aws_subnet" "private1" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = var.private_subnet1_cidr
  availability_zone  = "ap-south-1a"
}

Help  Write Out  Where Is  Cut  Execute  Location  Undo  Set Mark  To Bracket  Previous
CloudShell  Feedback  Console Mobile App  © 2026, Amazon Web Services, Inc. or its affiliates.  Privacy  Terms  Cookie preferences
```

- variables.tf – stores variable definitions

```

aws
[Search] [Alt+S] Ask Amazon Q
GNU nano 6.2 variables.tf
variable "region" {
  default = "ap-south-1"
}

variable "vpc_cidr" {
  default = "10.0.0.0/16"
}

variable "public_subnet1_cidr" {
  default = "10.0.1.0/24"
}

variable "public_subnet2_cidr" {
  default = "10.0.2.0/24"
}

variable "private_subnet1_cidr" {
  default = "10.0.3.0/24"
}

variable "private_subnet2_cidr" {
  default = "10.0.4.0/24"
}

variable "instance_type" {
  default = "t3.micro"
}

variable "db_username" {}
variable "db_password" {}

^G Help ^O Write Out ^W Where Is ^X Cut ^G Execute ^C Location ^U Undo ^M-A Set Mark ^M-I To Bracket ^M-Q Previous
CloudShell Feedback Console Mobile App © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

```

- terraform.tfvars – stores variable values

```

aws
[Search] [Alt+S] Ask Amazon Q
GNU nano 6.2 terraform.tfvars
region = "ap-south-1"

vpc_cidr = "10.0.0.0/16"

public_subnet1_cidr = "10.0.1.0/24"

public_subnet2_cidr = "10.0.2.0/24"

private_subnet1_cidr = "10.0.3.0/24"

private_subnet2_cidr = "10.0.4.0/24"

instance_type = "t3.micro"

db_username = "admin"

db_password = "password123"

```

- outputs.tf – displays important output

```

aws
[Search] [Alt+S] Ask Amazon Q
GNU nano 6.2 output.tf
output "ec2_public_ip" {
  value = aws_instance.web.public_ip
}

output "load_balancer_dns" {
  value = aws_lb.alb.dns_name
}

output "rds_endpoint" {
  value = aws_db_instance.db.endpoint
}

```

STEP 5: Initialize Terraform

- Initialize the Terraform working directory.
- terraform init

```
ubuntu@ip-172-31-32-58:~/terraform-project$ terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.35.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

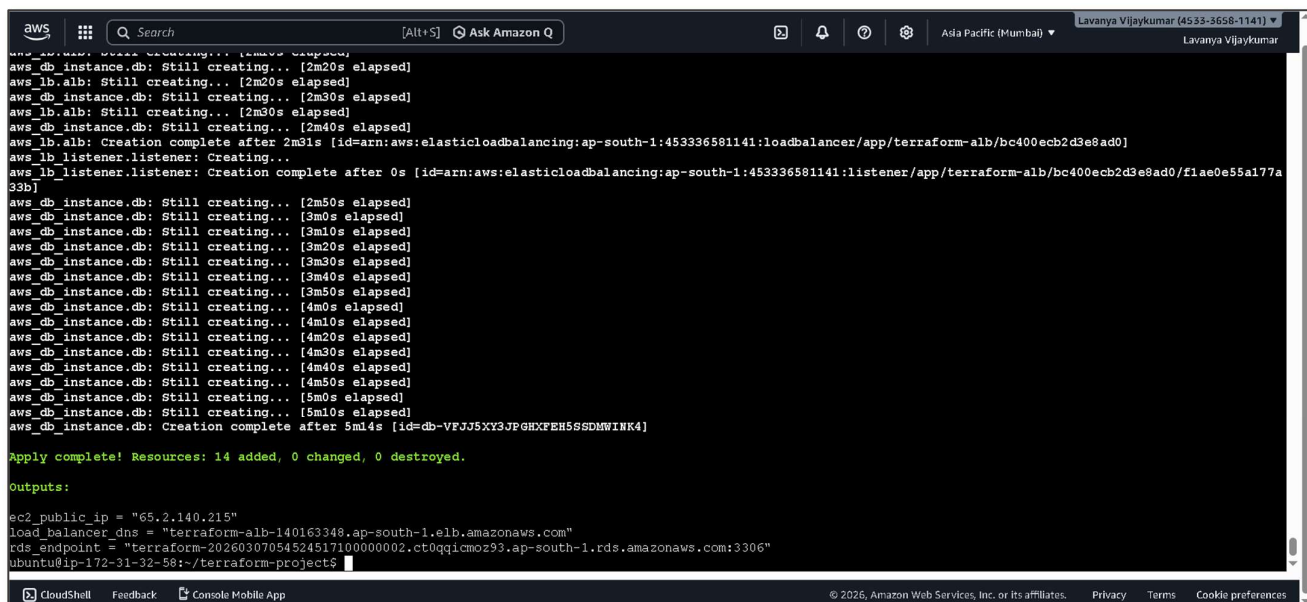
STEP 6: Format and Validate Terraform Configuration

- Check whether the Terraform files are correctly written.
- terraform validate ,terraform fmt

```
ubuntu@ip-172-31-32-58:~/terraform-project$ terraform fmt
ubuntu@ip-172-31-32-58:~/terraform-project$ terraform validate
Success! The configuration is valid.
```

STEP 7: Generate Execution Plan and Deploy Infrastructure

- Preview the infrastructure that Terraform will create, Apply the Terraform configuration to create AWS resources.
- terraform plan
- terraform apply
- Type yes when prompted.



```
aws
[Alt+S] Ask Amazon Q
Lavanya Vijaykumar (4533-3658-1141)
Lavanya Vijaykumar

aws_db_instance.db: Still creating... [2m20s elapsed]
aws_lb_alb: Still creating... [2m20s elapsed]
aws_db_instance.db: Still creating... [2m30s elapsed]
aws_lb_alb: Still creating... [2m30s elapsed]
aws_db_instance.db: Still creating... [2m40s elapsed]
aws_lb_alb: Creation complete after 2m31s [id=arn:aws:elasticloadbalancing:ap-south-1:453336581141:loadbalancer/app/terraform-alb/bc400ecb2d3e8ad0]
aws_lb_listener.listener: Creating...
aws_lb_listener.listener: Creation complete after 0s [id=arn:aws:elasticloadbalancing:ap-south-1:453336581141:listener/app/terraform-alb/bc400ecb2d3e8ad0/f1ae0e55a177a33b]
aws_db_instance.db: Still creating... [2m50s elapsed]
aws_db_instance.db: Still creating... [3m0s elapsed]
aws_db_instance.db: Still creating... [3m10s elapsed]
aws_db_instance.db: Still creating... [3m20s elapsed]
aws_db_instance.db: Still creating... [3m30s elapsed]
aws_db_instance.db: Still creating... [3m40s elapsed]
aws_db_instance.db: Still creating... [3m50s elapsed]
aws_db_instance.db: Still creating... [4m0s elapsed]
aws_db_instance.db: Still creating... [4m10s elapsed]
aws_db_instance.db: Still creating... [4m20s elapsed]
aws_db_instance.db: Still creating... [4m30s elapsed]
aws_db_instance.db: Still creating... [4m40s elapsed]
aws_db_instance.db: Still creating... [4m50s elapsed]
aws_db_instance.db: Still creating... [5m0s elapsed]
aws_db_instance.db: Still creating... [5m10s elapsed]
aws_db_instance.db: Creation complete after 5m14s [id=db-VFJJ5XY3JPGHXFEH5SSDMWINK4]

Apply complete! Resources: 14 added, 0 changed, 0 destroyed.

Outputs:
ec2_public_ip = "65.2.140.215"
load_balancer_dns = "terraform-alb-140163348.ap-south-1.elb.amazonaws.com"
rds_endpoint = "terraform-20260307054524517100000002.ct0q1cm0z93.ap-south-1.rds.amazonaws.com:3306"
ubuntu@ip-172-31-32-58:~/terraform-project$
```


STEP 8: Output will be EC2 instance, VPC, load balancer and RDS

The screenshot displays the AWS Management Console interface for an EC2 instance. The left sidebar shows the navigation menu with categories like EC2, Images, and Elastic Block Store. The main content area is titled "Instance summary for i-0795f741cf7ea4c1a". It provides a comprehensive overview of the instance's configuration and status.

Property	Value
Instance ID	i-0795f741cf7ea4c1a
Public IPv4 address	65.2.140.215 open address
Private IPv4 addresses	10.0.1.57
Instance state	Running
Public DNS	-
IPv6 address	-
Hostname type	IP name: ip-10-0-1-57-ap-south-1.compute.internal
Private IP DNS name (IPv4 only)	ip-10-0-1-57.ap-south-1.compute.internal
Instance type	t3.micro
Elastic IP addresses	-
Answer private resource DNS name	-
Auto-assigned IP address	65.2.140.215 [Public IP]
VPC ID	vpc-0982f633cd3874b9f
AWS Compute Optimizer finding	Opt-in to AWS Compute Optimizer for recommendations. Learn more
IAM role	-
Subnet ID	subnet-05c26527c9080c713
Auto Scaling Group name	-
IMDSv2	Optional
Instance ARN	arn:aws:ec2:ap-south-1:453336581141:instance/i-0795f741cf7ea4c1a
Managed	false

The screenshot shows the AWS Management Console's VPC dashboard. The left sidebar lists various VPC-related resources like Subnets, Route tables, and Internet gateways. The main content area displays "Your VPCs" with a table listing available VPCs. Below the table, the details for the selected VPC (vpc-0982f633cd3874b9f) are shown.

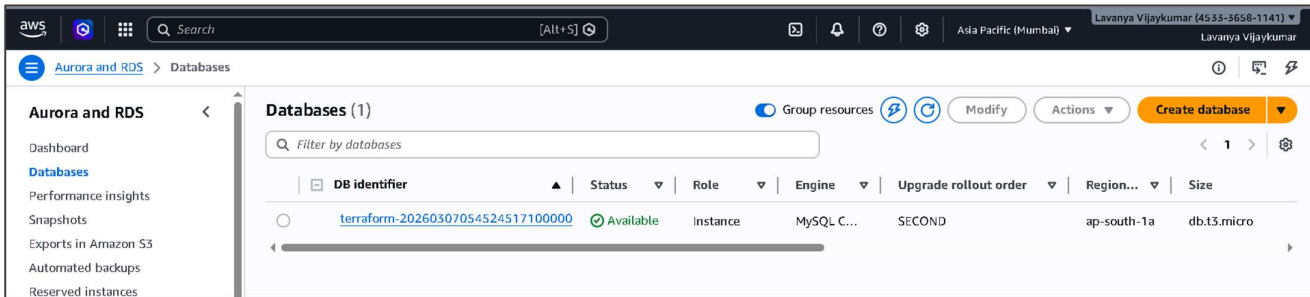
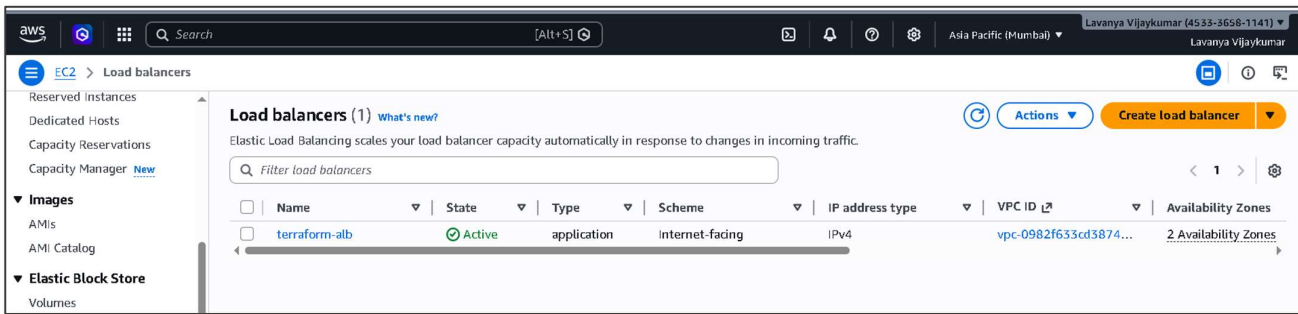
Name	VPC ID	State	Encryption ...	Encryption control ...	Block Public...	IPv
-	vpc-011fdadf19edca2a7	Available	-	-	Off	172
-	vpc-0982f633cd3874b9f	Available	-	-	Off	10.

vpc-0982f633cd3874b9f Details:

- VPC ID:** vpc-0982f633cd3874b9f
- State:** Available
- Block Public Access:** Off
- DNS hostnames:** Disabled
- DNS resolution:** Enabled
- Tenancy:** default
- DHCP option set:** dopt-037c89200ce8c1854
- Main route table:** rtb-01795d9d4cf6896fc
- Main network ACL:** acl-032033330f5c7b32f
- Default VPC:** vpc-011fdadf19edca2a7
- IPv4 CIDR:** 10.0.0.0/16
- IPv6 pool:** -

The screenshot displays the "Resource map" tab for the VPC vpc-0982f633cd3874b9f. This visual representation shows the hierarchical structure of the VPC, including its subnets and route tables.

- VPC:** Your AWS virtual network (vpc-0982f633cd3874b9f)
- Subnets (4):** Subnets within this VPC
 - ap-south-1a: subnet-05c26527c9080c713, subnet-06a29e7ef57dc7503
 - ap-south-1b: subnet-0eef9dfb83f272f4c, subnet-027abc9ce86d79355
- Route tables (1):** Route network traffic to resources (rtb-01795d9d4cf6896fc)



Conclusion:

This project successfully demonstrates how cloud infrastructure can be automated using Terraform on Amazon Web Services. By using Infrastructure as Code (IaC), the required resources such as Amazon VPC, Amazon EC2, AWS Elastic Load Balancing, and Amazon RDS were automatically provisioned and configured.

The implementation provides a secure and scalable architecture by deploying application servers in a public subnet and placing the database in a private subnet for better security. The load balancer ensures efficient distribution of incoming traffic, improving the availability and reliability of the application.

Overall, this project highlights the benefits of automation, consistency, and efficiency in cloud infrastructure management. Using Terraform simplifies the deployment process, reduces manual configuration errors, and enables repeatable infrastructure setups, making it an effective approach for modern cloud-based application deployments.